파이썬 프로그래밍

학습 내용 개요

파이썬 환경 설정 및 개발 도구

파이썬 역사, 철학 및 기본 개발 환경 설정

기초 문법 및 코드 구현

변수, 상수, 타입 힌트 및 핵심 문법 요소

함수와 제어문 심화

고급 함수 기법, 제어문, 반복문 활용법

객체지향 프로그래밍

클래스 정의, 상속, 다형성, 추상화 등 OOP 개념

파이썬의 역사와 철학

파이썬의 탄생

- 1991년 귀도 반 로섬(Guido van Rossum) 발표
- 이름 유래: BBC 코미디 "Monty Python's Flying Circus"
- 철학: 읽기 쉽고 배우기 쉬운 언어 → 유지보수, 협업에 강점

현재 위상

- 다목적 언어: 데이터 과학, 웹 개발, 자동화, AI 등 활용
- 방대한 생태계: PyPI 61만 개 이상 패키지 보유

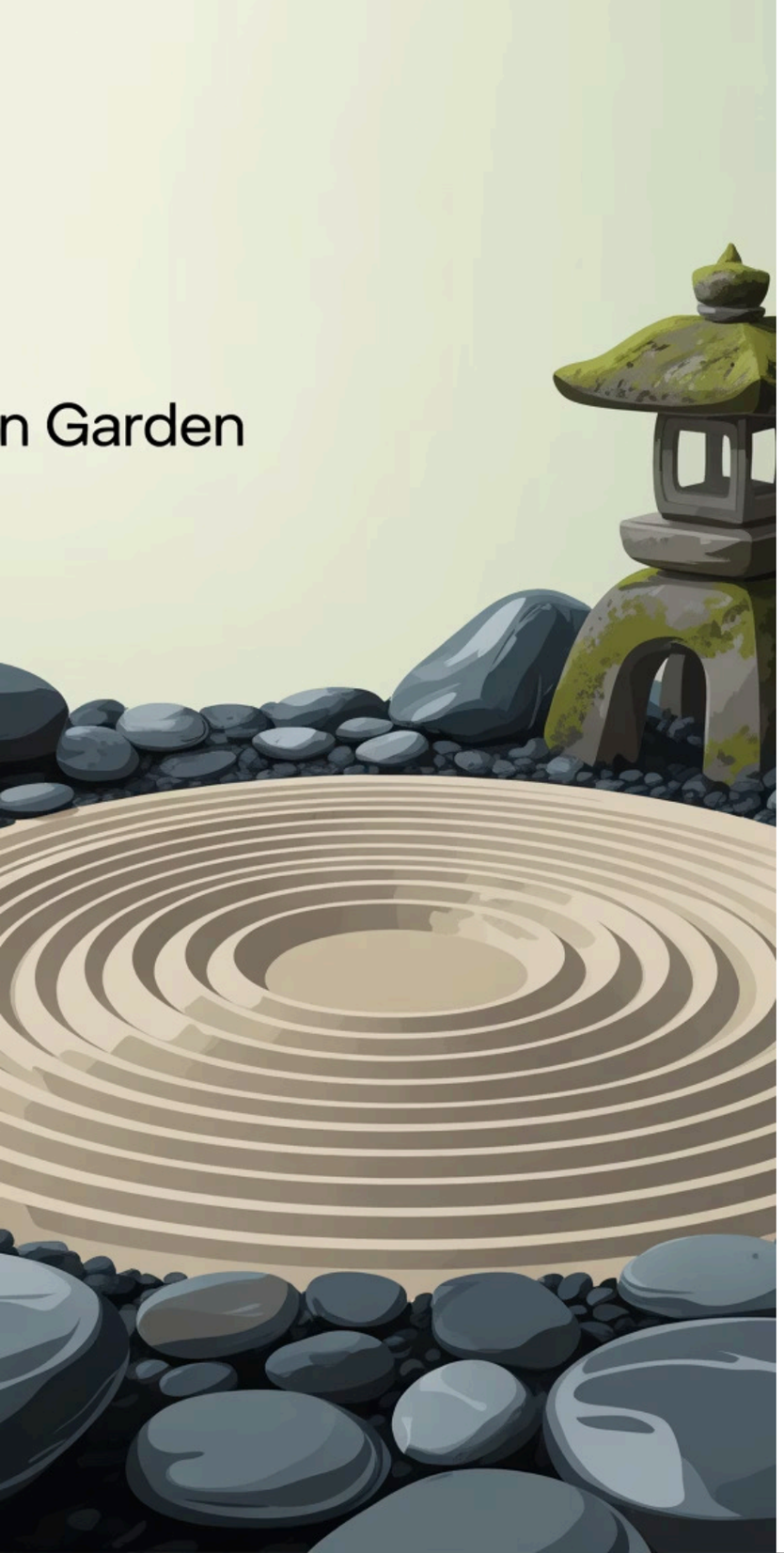
파이썬 인기 및 성장

TIOBE 인덱스 기준 파이썬 인기 지속 증가 추세 (2025년 5월, 점유율 25.35%로 최고 기록)

PYPL 지수: 최근 기준 Python 30.5% 점유율로 1위 언어 유지

Worldwide, Aug 2025 :

Rank	Change	Language	Share	1-year trend
1		Python	30.5 %	+0.9 %
2		Java	15.54 %	+0.2 %
3	↑↑	C/C++	8.3 %	+1.8 %
4	↓	JavaScript	7.32 %	-1.0 %
5	↓	C#	5.32 %	-1.3 %
6		R	5.19 %	+0.5 %
7	↑↑↑↑	Objective-C	3.57 %	+1.2 %
8	↓	PHP	3.49 %	-0.8 %
9	↑	Rust	2.63 %	-0.0 %
10	↓↓	TypeScript	2.48 %	-0.5 %
11	↓↓	Swift	2.17 %	-0.5 %
12	↑↑↑↑↑	Ada	1.74 %	+0.8 %
13	↓	Go	1.74 %	-0.4 %
14	↓	Kotlin	1.51 %	-0.4 %
15	↓	Matlab	1.5 %	+0.1 %



파이썬의 철학

> Beautiful is better than ugly.
>
> Explicit is better than implicit.
>
> Simple is better than complex.
>
> Readability counts.

`import this` 입력 시 파이썬 철학(PEP 20) 확인.

파이썬 설계 철학의 핵심

명확함 우선 (Explicit is better than implicit)

- 코드 동작 직관적 이해 강조
- 숨겨진 규칙보다 즉시 이해되는 문법 선호

가독성 중시 (Readability counts)

- 들여쓰기 강제로 코드 구조 명확
- 협업 시 타 개발자 읽고 수정 용이

단순함 지향 (Simple is better than complex)

- 불필요한 복잡성 최소화
- 문제 해결에 필요한 최소 문법과 도구 제공.

일관성 유지 (Consistency matters)

- 표준 라이브러리와 문법 스타일 일관성 유지
- PEP 8 가이드로 코드 품질 보장

파이썬 기초 문법: 변수와 상수

변수(Variable)

- 메모리에 데이터 저장하는 이름
- 동적 타이핑(Dynamic Typing): 타입 변경 가능
- 예시: `counter = 10`, `counter = "ten"` (타입 변경)

```
# 변수 예시
name = "홍길동"  # 문자열
age = 25       # 정수
is_student = True  # 불리언
```

상수(Constant)

- 값 한 번 정하면 변경 않음
- Python, 언어 차원 상수 키워드 없음
- 관례: 대문자 변수명 사용

```
# 상수 예시
PI = 3.14159
MAX_CONNECTIONS = 100

# typing.Final 사용 (Python 3.8+)
from typing import Final
TAX_RATE: Final = 0.1
```

타입 힌트(Type Hinting)

파이썬 3.5+ 타입 힌트 도입. 코드 가독성 향상, IDE 자동완성 강화.

```
def calculate_area(radius: float) -> float:
    """원의 면적을 계산합니다."""
    return 3.14159 * radius * radius

def greet(name: str, age: int = 20) -> str:
    return f"안녕하세요, {name}님! {age}세이시네요."
```

mypy 등 정적 분석 도구, 타입 오류 사전 감지 가능.

실습: 데이터클래스로 상품 관리

데이터클래스 활용: 상품 정보 구조화 및 재고 가치 계산 함수 구현

- 데이터클래스: 데이터 중심 클래스 생성 간소화
- 타입 힌트와 결합, 명확한 코드 작성
- 총 재고 가치 계산 함수 구현

```
from dataclasses import dataclass
from typing import List

@dataclass
class Product:
 name: str
 price: float
 quantity: int = 0

 def total_value(self) -> float:
 return self.price * self.quantity

def calculate_inventory_value(products: List[Product]) -> float:
 return sum(p.total_value() for p in products)
```

```
# 사용 예시
products = [
 Product("노트북", 1200000, 5),
 Product("마우스", 30000, 20),
 Product("키보드", 100000, 10)
]

total = calculate_inventory_value(products)
print(f"총 재고 가치: {total:,}")
```

리스트 컴프리헨션(List Comprehension)

정의 및 장점

- 기존 리스트 변환/필터링 후 새 리스트 생성
- 코드 간결성: 여러 줄 for문 → 한 줄로
- 속도 향상: 일반 for문 대비 빠름

문법

```
[표현식 for 항목 in 반복가능객체 if 조건식]
```

```
# 1~10 중 짝수 제곱 리스트 생성
squares = [x**2 for x in range(1, 11) if x % 2 == 0]
print(squares) # [4, 16, 36, 64, 100]

# 중첩 리스트 컴프리헨션
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flattened = [num for row in matrix for num in row]
print(flattened) # [1, 2, 3, 4, 5, 6, 7, 8, 9]

# 셋 컴프리헨션
unique_letters = {c.lower() for c in "Hello World"}
print(unique_letters) # {'h', 'e', 'l', 'o', 'w', 'r', 'd', ' '}
```

딕셔너리 컴프리헨션

반복문을 활용하여 딕셔너리를 간결하게 생성.

문법

```
{키표현식: 값표현식 for 항목 in 반복가능객체 if 조건식}
```

예시

```
# 숫자:제곱 값 딕셔너리 생성
square_dict = {x: x**2 for x in range(1, 6)}
print(square_dict) # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# 기존 딕셔너리에서 특정 조건의 항목만 필터링
prices = {"apple": 1000, "banana": 2000, "orange": 1500, "grape": 3000}
expensive = {k: v for k, v in prices.items() if v > 1500}
print(expensive) # {'banana': 2000, 'grape': 3000}
```

유용한 내장 함수: zip(), enumerate()

zip() 함수

여러 반복가능객체 병렬 순회 시 사용.

```
names = ["Alice", "Bob", "Charlie"]
scores = [85, 92, 78]

for name, score in zip(names, scores):
    print(f"{name}: {score}")
# 출력:
# Alice: 85
# Bob: 92
# Charlie: 78
```

enumerate() 함수

반복 시 인덱스와 값 동시 반환.

```
items = ["사과", "바나나", "체리"]

for idx, value in enumerate(items,
start=1):
 print(f"{idx}번째 항목: {value}")
# 출력:
# 1번째 항목: 사과
# 2번째 항목: 바나나
# 3번째 항목: 체리
```

유용한 내장 함수: map(), filter(), lambda

map() 함수

반복 가능한 객체의 각 요소에 함수를 적용, 새 이터레이터 반환

```
numbers = [1, 2, 3, 4]
squared = list(map(lambda x: x**2, numbers))
print(squared)  # [1, 4, 9, 16]
```

filter() 함수

조건에 맞는 요소만 필터링

```
numbers = [10, 15, 20, 25]
even_numbers = list(filter(lambda x: x % 2 == 0,
numbers))
print(even_numbers)  # [10, 20]
```

lambda 함수

한 줄 익명 함수. **map()**, **filter()** 등과 주로 함께 사용.

```
# 기본 문법: lambda 매개변수: 표현식
add = lambda x, y: x + y
print(add(3, 4)) # 7

# 정렬에 활용
students = [
 {"name": "김철수", "score": 85},
 {"name": "이영희", "score": 92},
 {"name": "박민수", "score": 78}
]
students.sort(key=lambda s: s["score"],
reverse=True)
print(students) # 점수 내림차순 정렬
```

실습: 학생 성적 분석 프로그램

```
# 학생 정보와 성적 데이터
students = ["김철수", "이영희", "박민수", "정지원",
"최유진"]
scores = {"국어": [85, 92, 78, 90, 85],
 "영어": [82, 88, 95, 70, 93],
 "수학": [78, 96, 85, 92, 88]}

# enumerate, zip 활용
for idx, name in enumerate(students, 1):
 kor, eng, math = [scores[subject][idx-1] for
subject in ["국어", "영어", "수학"]]
 avg = (kor + eng + math) / 3
 print(f"{idx}번 {name}: 평균 {avg:.1f}점")
```


```
# 학생 정보와 성적 데이터
students = ["김철수", "이영희", "박민수", "정지원",
"최유진"]
scores = {"국어": [85, 92, 78, 90, 85],
 "영어": [82, 88, 95, 70, 93],
 "수학": [78, 96, 85, 92, 88]}

# enumerate, zip 활용
for idx, name in enumerate(students, 1):
 kor, eng, math = [scores[subject][idx-1] for
subject in ["국어", "영어", "수학"]]
 avg = (kor + eng + math) / 3
 print(f"{idx}번 {name}: 평균 {avg:.1f}점")
```

함수 심화: 가변 인자 활용

*args / **kwargs

- `*args`: 위치 인자를 튜플 형태로 전달
- `**kwargs`: 키워드 인자를 딕셔너리 형태로 전달
- 함수 호출 시 `*`/`**`로 시퀀스·맵 해제 가능

```
def calculate_stats(*numbers, **options):
 """여러 숫자의 통계를 계산합니다."""
 result = {}

 if "average" in options and options["average"]:
 result["평균"] = sum(numbers) / len(numbers)

 if "max" in options and options["max"]:
 result["최대값"] = max(numbers)

 return result

# 호출 예시
stats = calculate_stats(10, 20, 30, 40, average=True, max=True)
print(stats) # {'평균': 25.0, '최대값': 40}
```

함수 심화: 람다, 중첩 함수

람다(lambda) 표현식

- 간단한 익명 함수 (lambda x: x*2)
- 한 줄 간결 표현
- 주로 정렬, 필터링에 활용

```
# 리스트의 튜플을 값 기준으로 정렬
nums = [("a", 3), ("b", 1), ("c", 5)]
nums.sort(key=lambda x: x[1])
print(nums)  # [('b', 1), ('a', 3), ('c', 5)]
```

중첩 함수(Inner Function)

- 외부 함수 내 사용 헬퍼 함수
- 장점: 코드 지역화, 가독성/재사용성 향상

```
def normalize(data):
 """데이터를 0~1 범위로 정규화합니다."""
 def scale(x):
 return (x - mn) / (mx - mn)

 # 최소값과 최대값은 한 번만 계산
 mn, mx = min(data), max(data)
 return [scale(x) for x in data]

# 사용 예시
scores = [72, 85, 93, 65, 99]
norm_scores = normalize(scores)
print(norm_scores)
```

클로저(Closure)

상태를 가진 함수 만들기

클로저는 외부 스코프 변수를 캡처해 저장하는 함수.

`nonlocal` 키워드 사용 시 외부 함수 변수 수정 가능.

```
def counter(start=0, step=1):
 """증가하는 카운터 함수 반환."""
 n = start

 def inc():
 nonlocal n
 n += step
 return n

 return inc
```

```
# 사용 예
c1 = counter(start=10, step=5)
print(c1(), c1(), c1()) # 15, 20, 25

c2 = counter() # 기본값 사용
print(c2(), c2()) # 1, 2
```

데코레이터(Decorator)

함수 기능 확장하기

- 함수 인자로 받아 새 함수 반환
- 로깅, 측정, 검증, 캐시 등 활용
- `functools.wraps`로 메타데이터 보존

```
from functools import wraps
import time

def timed(fn):
 """함수 실행 시간 측정 데코레이터"""
 @wraps(fn)
 def wrapper(*args, **kwargs):
 start = time.time()
 result = fn(*args, **kwargs)
 end = time.time()

 print(f"{fn.__name__} 함수 실행 시간: {end-start:.4f}초")
 return result

 return wrapper
```

```
@timed
def slow_function(n):
 """시간 소요 함수"""
 time.sleep(n) # n초 대기
 return f"{n}초 기다렸습니다!"

# 사용 예시
print(slow_function(1))
# slow_function 함수 실행 시간: 1.0012초
# 1초 기다렸습니다!
```

매개변수 데코레이터

데코레이터에 매개변수 전달 시, 한 단계 더 감싸는 함수 필요.

```
from functools import wraps

def retry(times=3, exceptions=(Exception,)):
    def outer(fn):
        @wraps(fn)
        def wrapper(*a, **k):
            for attempt in range(1, times + 1):
                try:
                    return fn(*a, **k)
                except exceptions as e:
                    if attempt == times:
                        raise
        return wrapper
    return outer

@retry(times=5)
def flaky(): ...
```

제어문 심화: match 패턴 매칭

Python 3.10+에서 도입된 구조적 패턴 매칭. 다중 조건 처리를 더 직관적으로 수행.

기본 패턴 매칭

```
def check_status(status):
    match status:
        case 200:
            return "OK"
        case 404:
            return "Not Found"
        case 500:
            return "Server Error"
        case _:  # 기본 케이스
            return f"Unknown Status: {status}"

# 사용 예시
print(check_status(404))  # "Not Found"
```

구조 분해 패턴 매칭

```
def process_data(data):
 match data:
 case {"type": "user", "name": name, "age": age}:
 return f"사용자 {name}님은 {age}세입니다."

 case {"type": "product", "name": name, "price": price}:
 return f"제품 {name}의 가격은 {price}원입니다."

 case [x, y, z]:
 return f"좌표: ({x}, {y}, {z})"

 case _:
 return "알 수 없는 데이터 형식입니다."

# 다양한 데이터로 테스트
print(process_data({"type": "user", "name": "김철수", "age": 25}))
print(process_data([10, 20, 30]))
```

고급 예외 처리 구조

try 블록

예외 발생 가능 코드 실행.

```
try:
    num = int(input("숫자 입력: "))
    result = 100 / num
```

except 블록

예외 발생 시 실행.

```
except ValueError:
    print("유효한 숫자가 아닙니다.")
except ZeroDivisionError:
    print("0으로 나눌 수 없습니다.")
except Exception as e:
    print(f"오류 발생: {e}")
```

else 블록

예외 미발생 시 실행.

```
else:
    print(f"결과: {result}")
```

finally 블록

예외 발생 여부 무관, 항상 실행.

```
finally:
 print("프로그램 종료")
```

이터레이터와 제너레이터

이터레이터(Iterator)

- 순회 가능한 객체
- `__iter__()`, `__next__()` 메서드 구현
- 요소 하나씩 처리 (메모리 효율적)

```
class Counter:
    def __init__(self, max):
        self.max = max
        self.num = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.num >= self.max:
            raise StopIteration
        self.num += 1
        return self.num

# 사용 예시
for num in Counter(5):
    print(num)  # 1, 2, 3, 4, 5
```

제너레이터(Generator)

- `yield`로 값 하나씩 반환
- 함수 상태 기억 (일시 정지/재개 가능)
- 지연 평가: 필요 시 계산

```
def countdown(n):
 """n부터 1까지 카운트다운하는 제너레이터"""
 while n > 0:
 yield n
 n -= 1

# 사용 예시
for num in countdown(5):
 print(num) # 5, 4, 3, 2, 1

# 무한 시퀀스 생성 가능
def infinite_counter(start=0):
 while True:
 yield start
 start += 1
```

객체지향 프로그래밍 - 기본 개념

캡슐화(Encapsulation)

데이터와 관련 메서드 한 단위로 묶기

예: 클래스 변수/메서드, 접근 제어

추상화(Abstraction)

복잡한 내부 숨기고 필요 인터페이스만 노출

예: 추상 클래스, 인터페이스

상속(Inheritance)

기존 클래스 속성/메서드 재사용

예: 부모 Vehicle, 자식 Car/Bicycle

다형성(Polymorphism)

동일 인터페이스로 다양한 구현체 접근

예: speak() 메서드, 동물마다 다르게 구현

객체지향 프로그래밍: 코드 재사용성 및 유지보수성 향상.

상속과 다형성

상속(Inheritance)

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print("동물이 소리를 냅니다")

class Dog(Animal):
    def speak(self):  # 메서드 오버라이딩
        print(f"{self.name}가 멍멍 짖습니다")

class Cat(Animal):
    def speak(self):
        print(f"{self.name}가 야옹 웁니다")

# 부모 메서드 호출
class Puppy(Dog):
    def speak(self):
        super().speak()  # 부모 메서드 호출
        print("꼬마 강아지 소리도 냅니다")
```

다형성(Polymorphism)

```
def make_speak(animal):
 """어떤 동물 객체든 speak 메서드 호출"""
 animal.speak()

# 서로 다른 클래스 객체
dog = Dog("바둑이")
cat = Cat("나비")

# 동일 인터페이스 호출
make_speak(dog) # 바둑이가 멍멍 짖습니다
make_speak(cat) # 나비가 야옹 웁니다

# 동적 바인딩
animals = [Dog("멍멍이"), Cat("야옹이"), Puppy("솜이")]
for animal in animals:
 animal.speak() # 각 동물 타입에 맞는 메서드 실행
```

추상 클래스 & 인터페이스

추상 클래스: 구현 강제, 일관된 인터페이스 보장.

```
from abc import ABC, abstractmethod

class Payment(ABC):
 """결제 방식에 대한 추상 클래스"""

 @abstractmethod
 def pay(self, amount):
 """결제를 처리합니다."""
 pass

 @abstractmethod
 def refund(self, amount):
 """환불을 처리합니다."""
 pass
```

```
class CreditCardPayment(Payment):
 def pay(self, amount):
 print(f"신용카드로 {amount}원 결제")
 return True

 def refund(self, amount):
 print(f"신용카드로 {amount}원 환불")
 return True

# 추상 메서드를 모두 구현하지 않으면 오류 발생
# class IncompletePayment(Payment):
# def pay(self, amount):
# pass
# # refund() 메서드가 없어서 인스턴스화 불가
```

실습: 회원 관리 프로그램 만들기

프로그램 실행 상태에서

콘솔 메뉴: 1) 회원 등록 2) 회원 목록 3) 활성/비활성 전환 0) 종료

- 등록 시 ID 자동 증가
- 목록에 모든 회원 표시
- 상태 토글 정상 작동
- 잘못된 입력/없는 ID → 오류 메시지
- 빈 값 입력 → 오류 메시지
- 종료 시 에러 없이 끝남